

Analysis: Contranmutation

Test Set 1

In Test Set 1 everything is small. One possible solution is to simulate the process. There is never a reason to not convert elements other than lead, so we should just apply the transformations over and over until no more lead can be created. If after a long time the amount of lead keeps growing, that means the total is unbounded. In the Test Set 2 section we do a formal analysis on bounds, but for this one, using an intuitive definition of "a lot" should suffice. If the amount is bounded, 1 gram of a metal can transform into at most 512 grams of lead, since it can only go through 9 splits before getting into a cycle. That means if we ever see more than 5120 grams of lead, the amount is unbounded.

However, the first example shows that we need to do a little more. What if lead itself can be used to get more lead? If that's possible, and we can get any lead at all, the final amount is definitely unbounded. If it is not possible, then we have no reason to ever consume lead. One way to check is to do the simulation above starting with 1 gram each of the two elements that lead is turned into. If that leads to 0 or 1 grams of lead, then lead cannot be used to get more lead. Otherwise, it can.

Test Set 2

Test Set 2 is a lot larger, so a hand-wavy analysis and implementation will not work. However, we can use the same idea above more carefully. Since each metal takes at most M steps to turn into lead, we only need to do M iterations of a simulation, where a simulation involves going over all non-lead metals and converting all grams of them. This takes $O(M^2)$ time. To prevent the numbers from growing too big, we can simply do this process twice: the first pass creates as much lead as possible from non-cyclical sources, so if the second pass creates even more, then it must come from cyclical sources that ultimately yield an unbounded amount of lead. The second part, to check for lead creating more lead, is the same as before.

Notice that the result does not fit in 64 bit integers, though, and doing modulo all the time as usual doesn't work right away, since we cannot afford to equate "no X" with " 10^9+7 grams of X" if X has the capability of creating unbounded amounts of lead (with X being lead itself, or some other metal). There are multiple ways to get around the problem, including using big integers, storing an additional bit for each result that represents whether it is an actual 0 value, and doing calculations modulo $10^9+7 \times K$ for some large random prime K until the very end.

There are other solutions that only work for Test Sets 1 and 2, that are less efficient implementations of the solution for Test Set 3, that is explained below.

Test Set 3

For Test Set 3, the solution looks quite different but it is actually similar. We check for cycles that may generate unbounded lead, and if there are none, we can do the simulation with a single iteration, ordering the metals appropriately. All of those things can be done in linear time, and we describe how below.

One useful thing to notice early on is that if we can determine that the amount of lead we can produce is bounded, the only formulas that are worth using are those that can eventually (perhaps in conjunction with other formulas) produce lead. Moreover, if the amount of lead is

bounded, there is no point in using lead as an input to any formula, because even in the best case, every unit of lead will turn into one unit of lead and one unit of something else that cannot be converted to lead. (Otherwise, we could simply apply that series of formulas and generate as much lead as we want).

These observations imply that we can only get an unlimited amount of lead if some metal can produce itself and lead at the same time. In other words, there must be a series of formulas that takes one unit of some metal X and turns it into one unit of X and one unit of lead. In the process, we may end up with some other metals as well, but those don't matter. Notice that if that metal X directly produces metals A and B , then we require that either A produces X and B produces lead, or vice versa. This is because even if X is lead itself and either of A or B is also lead, without considering the other result, we are just destroying 1 unit of lead to get 1 unit of lead, so the total amount of lead doesn't change. In other words, in the unbounded case, after applying each formula, we need to work on both of the outputs, not just one of them.

Let Q be a graph in which each metal is represented by a node, and there is an edge from node u to node v if, by consuming 1 unit of u , we can produce 1 unit of v . Notice that if a metal i is initially not present (that is, $G_i = 0$), and it is not reachable in Q from some j with $G_j > 0$, then it is impossible to produce metal i . Therefore, we can remove any such metals at the outset. In what follows, we assume that Q contains no such metals.

Let Q' be the [transpose](#) of Q , and let L denote the node representing lead. Then, traversing Q' from node L defines a subgraph Q'_L , where every node is a metal that can produce lead after a series of transformations. Now, let P_u be the number of simple paths (paths without cycles) from L to any other node u in Q'_L . If the amount of lead is bounded, then P_u is the number of units of lead we can get from every unit of u . This follows from the fact that every (reversed) edge is a valid transformation where we start with 1 unit of some metal and produce 1 unit of some other metal, so along the path, we are using the results of previous transformations and not the very first unit of metal we started with. For example, let Q'_L be the following graph:

- $1 \rightarrow 2$
- $1 \rightarrow 3$
- $2 \rightarrow 4$
- $3 \rightarrow 4$

Which means the valid transformations are:

- 4 turns into 2 and 3
- 2 turns into 1 and something else
- 3 turns into 1 and something else

There are 2 simple paths from 1 to 4, namely, $1 \rightarrow 2 \rightarrow 4$ and $1 \rightarrow 3 \rightarrow 4$. Now, 1 unit of metal 4 can be turned into 2 units of lead (1) as follows:

- Turn one unit of 4 into one unit of 2 and one unit of 3
- Take the one unit of 3 from step 1 and turn it into one unit of 1 and one unit of some other metal
- Take the one unit of 2 from step 1 and turn it into one unit of 1 and one unit of some other metal

On the other hand, notice that the nodes that form a [strongly connected component \(SCC\)](#) in Q are a set of metals that can produce each other. More formally, if u and v belong to the same SCC, we can produce 1 unit of v from 1 unit of u , and vice versa. With these definitions, we can say that the amount of lead is unbounded if, for some node u in Q with v_1 and v_2 being the two outputs of node u 's formula, either:

- v_1 is in Q'_L and v_2 and u are in the same SCC, or
- v_2 is in Q'_L and v_1 and u belong to the same SCC.

Otherwise, there is a limit on the amount of lead we can get, which can be computed by multiplying P_u by the initial number of units G_u for each u in Q'_L .

We can compute all the SCCs in $O(M)$ time using [Tarjan's algorithm](#) or [Kosaraju's algorithm](#), since both the number of nodes and the number of edges in the graph are linear on M . Walking over the graphs defined above also takes $O(M)$ time, and so does checking the unbounded condition. We can also compute all P_u in $O(M)$ time by taking advantage of the fact that if the amount of lead is not unbounded, then there are no cycles in Q'_L . (If a cycle existed in Q'_L , then by reversing the edges, we get at least one metal for which the unbounded condition holds.) Therefore, the nodes on Q'_L can be topologically sorted. That means we can compute a function F from vertices to the number of simple paths in linear time, using dynamic programming with this recursive definition:

- $F(L) = 1$
- $F(u) = \sum(F(v))$ for each v , such that there is an edge from v to u in Q'_L

Since the number of edges in every graph described above is also $O(M)$, the whole problem can be solved in linear time.

Analysis: New Elements: Part 1

Test set 1

In the first test set, there can be at most six molecules, so we can easily check all $6! = 720$ possible orderings of them. The nontrivial part is figuring out how to check an ordering. We cannot check all possible atomic weights, since they can be arbitrary integers, and those integers might need to be quite large (since the C_i and J_i values can be as large as 10^9).

For each ordering of molecules, we must determine whether there is at least one valid pair of atomic weights w_C and w_J for Codium and Jamarium, respectively, such that the molecules are in strictly increasing order of molecular weight. We do not need to find particular values; we only need to show existence or non-existence.

Let $k = w_J / w_C$. Then, if w_C and w_J are valid for our ordering, we can substitute $w_J = k \times w_C$ in the requirement for any pair of molecules (C_a, J_a) , (C_b, J_b) such that $a < b$:

$$C_a \times w_C + J_a \times k w_C < C_b \times w_C + J_b \times k w_C$$

If $J_a = J_b$, then this reduces to $C_a < C_b$, which is a simple check; if it is false, then no set of atomic weights is valid, and we can stop checking this ordering. Otherwise, rearranging some terms and dividing, our expression becomes:

$$(C_a - C_b) / (J_b - J_a) < k, \text{ if } J_a < J_b, \text{ or} \\ k < (C_a - C_b) / (J_b - J_a), \text{ if } J_a > J_b.$$

So, we can start by assuming that k can take on any value in an infinite range. Then, whenever we check a pair of molecules, we get a new (non-inclusive) upper or lower bound on the value of k , and we can update one end of the range accordingly. If we find that constraints force the range to be empty (e.g., because the upper end is forced to be smaller than the lower end), then no valid set of atomic weights exists. Otherwise, at least one does. Notice that even though the atomic weights of Codium and Jamarium must both be integers, any rational value of k corresponds to some pair of integers.

This is a quick set of checks, and it is even quicker if we realize that we only need to check consecutive pairs of molecules. However, as in any problem involving comparisons of fractional quantities, we must be careful to compute exact values instead of using floating-point approximations.

Test set 2

Considering the solution of Test set 1 in reverse can lead us to a solution for Test set 2. If we consider two arbitrary molecules with indices a and b , they can appear in two possible orderings. If one of them is impossible according to the math above, it means that the other is the ordering of the molecules for every possible assignment of atomic weights. If that's the case, we can ignore the pair of molecules (a, b) . However, if both orderings are possible, we obtain a range of valid values for the ratio k of the form $(0, R_{a,b})$ for one ordering and a range of the form $(R_{a,b}, +\infty)$ for the other, where $R_{a,b} = |(C_a - C_b) / (J_a - J_b)|$. That means, for all orderings that correspond to atomic weights that yield a ratio strictly less than $R_{a,b}$, the molecules a and b appear in a specific relative order, and for all orderings that correspond to values yielding a ratio

strictly greater than $R_{a,b}$, the molecules a and b appear in the other relative order. If the ratio is exactly $R_{a,b}$, the two molecules weigh exactly the same, so no ordering is valid.

By the previous paragraph, if we consider a function f from ratios into orderings, f is a piecewise constant function that is undefined at ratios $R_{a,b}$ for any pair a, b and is constant in any interval that doesn't contain any such ratio. Moreover, the image of f over $(0, R_{a,b})$ consists of orderings with molecules a and b in one relative order and the image of f over $(R_{a,b}, +\infty)$ consists of orderings with molecules a and b in the other relative order. Therefore, no two elements in f 's image are the same. The number of elements in f 's image, which is the answer to the problem, is the number of different non-zero $R_{a,b}$ values, plus 1. Notice that it is possible that $R_{a,b} = R_{c,d}$ for different molecules a, b, c, d , and we need to count the number of unique values.

The algorithm to count the number of different values that are equal to $R_{a,b}$ for some a, b is immediate: try every pair of molecules, calculate the ratio as explained for the Test set 1 solution (if there is one), and add it to a set of fractions to discard non-uniques. This algorithm takes a quadratic number of insertions into a set, which makes it quadratic or barely above quadratic overall depending on the set implementation that is used.

Analysis: New Elements: Part 2

Let w_C be the atomic weight of Codium and w_J be the atomic weight of Jamarium according to the rules given in the problem statement. Let ΔC_i equal $C_{i+1} - C_i$ and ΔJ_i equal $J_{i+1} - J_i$, for all $1 \leq i < N$. As in our analysis for New Elements, Part 1, we have:

- $-\Delta C_i / \Delta J_i < w_J / w_C$, if $\Delta J_i > 0$.
- $w_J / w_C < -\Delta C_i / \Delta J_i$, if $\Delta J_i < 0$.
- $-\Delta C_i \times w_C < 0$, if $\Delta J_i = 0$.

Therefore, we can get the lower bound and upper bound of w_J / w_C just by looking at consecutive indices. We can initially set the lower bound (let us represent it with the reduced fraction L_N / L_D) to be 0 and the upper bound (let us represent it with the reduced fraction U_N / U_D) to be ∞ . We update either L_N / L_D or U_N / U_D for each pair of consecutive indexes, just as in our analysis from Part 1.

Once we have L_N / L_D and U_N / U_D , we want to find a rational number w_J / w_C such that $L_N / L_D < w_J / w_C < U_N / U_D$. If $L_N / L_D \geq U_N / U_D$, then there is certainly no solution. Otherwise, there must be at least one solution; for example, the [mediant](#) $(L_N + U_N) / (L_D + U_D)$ is certainly between the bounds. However, the problem asks us to minimize w_C and w_J (first w_C , and then w_J).

Test set 1

Since $\Delta J_i \leq 99$ in this test set, we get $L_D + U_D \leq 198$. Therefore, we know that a solution with $w_C \leq 198$ exists. We can try all possible values from 1 to 198 as w_C . For each choice of w_C , we can derive the smallest w_J such that $L_N / L_D < w_J / w_C$, and then we can check whether $w_J / w_C < U_N / U_D$.

Test set 2

For each integer C (from 1 to $L_D + U_D$), we can check whether there is a rational number that is strictly between L_N / L_D and U_N / U_D , and has a denominator that is not more than C . To do that, we can find a rational number with denominator not more than C closest to the average of L_N / L_D and U_N / U_D . This is because all rational numbers that are strictly between L_N / L_D and U_N / U_D are closer to the average of L_N / L_D and U_N / U_D than all rational numbers that are not strictly between L_N / L_D and U_N / U_D . We can do so by using a library function like Python's [fractions.limit_denominator](#), or by implementing our own approximation using [continued fractions](#).

Once we can solve the problem given in the previous paragraph, we can use binary search to find w_C as the smallest C such that a rational number with denominator not more than C , and strictly between L_N / L_D and U_N / U_D exists. Just as we did for the previous test set, we can derive the smallest w_J such that $L_N / L_D < w_J / w_C$.

Analysis: Pottery Lottery

A simple (but incorrect) approach

Our tempting first thought might be: why not just put one of our tokens in each vase? Then we can sit back and just wait to win, no matter which vase gets chosen. Who needs the other 80 days?

The flaw in this reasoning is that if multiple vases are tied for having the fewest tokens, there is no winner. A quick simulation can show us that there is only a unique winner around 57% of the time. So this approach has no hope of passing, and we have to work harder!

A slightly more complex (but also incorrect) approach

It might still seem that we can succeed without inspecting any vases. What if we just choose one vase up front — without loss of generality, we will pick vase #20 — and try to make it the winning vase? Over the course of the first 99 nights, we distribute fake tokens among the other 19 vases, as evenly as possible (to prevent any one of those vases from becoming more of a threat than the others). The labels on these fake tokens do not matter, since we are banking on vase 20 winning. Then, on the 100th night, we place our token in vase 20. Since each of the other 19 vases will be burdened with 5 (or maybe even 6) extra tokens, the odds are good that vase 20 will have the fewest, right?

Unfortunately, those odds aren't so good. Again, we can write a quick simulation to verify that our strategy will only succeed around 53% of the time — it's even worse than the strategy above! The average "burden" that we add to the other vases just isn't large enough compared to the random variation in the number of tokens per vase.

A better approach

We will need to take advantage of our inspection ability, but when should we use it? There is not much value in inspecting early on in the lottery, since relatively few tokens will have been placed. On the other hand, if we inspect too late, we may not have enough nights left on which to react to whatever information we discover.

We can devise a version of our first approach that also incorporates inspection. In this approach, V and N are parameters that we will have to discover:

1. Choose to "give up on" the first V vases. We will assume that none of these will win, and so the labels on the tokens we add to them will not matter. Spend the first N nights sabotaging them (evenly).
2. Spend the next 20 turns inspecting every vase, in order.
3. Based on the results of our inspections, choose the vase with the fewest tokens (out of the remaining $20 - V$ vases) to be our candidate winning vase.
4. On each of the remaining $99 - 20 - N$ nights, choose a vase (other than our candidate vase) with the smallest number of tokens in our inspection results, and add a token to it. Then, update the inspection results to reflect this.
5. On day 100, add our own token to the candidate vase.

We might worry that our inspections are only snapshots in time that grow stale. By the time we inspect vase 20, our estimate for vase 1 is already out-of-date — what if more tokens have been added to it since then? To compare the estimates a little more fairly, we could adjust them by the

expected number of tokens that have been added since the night of the estimate (i.e., the number of additional nights, divided by 20). This adjustment will be at most .95, which is less than a single full token, so the adjustments could only possibly be used to break ties among vases with the same number of tokens... and we can get the same effect by just choosing the vase with the highest-numbered ID in the event of a tie.

It is not too hard to simulate this process, so we can try out different values of V and N to see what yields the best results, and find that $V = 14$, $N = 60$ works about 95% of the time. That gives us, per the binomial distribution, about a 99.96% chance of solving at least 225 out of 250 cases correctly. Since the problem only has one Visible test set, we should just submit and hope for the best!

Can we do even better?

If we want to experiment further, there are even better solutions out there! Here is a variant of the above strategy; it succeeds over 98% of the time:

- Days 1-60: Put 4 tokens into each of vases 1-15.
- Days 61-80: Inspect every case. Find the *two* vases with the smallest number of tokens (breaking ties in favor of higher-numbered vases, as we will throughout this strategy), and designate them as candidates.
- Days 81-94: Greedily put a token into the non-candidate vase that has the smallest number of tokens, per our inspection results. Update those results.
- Days 95-96: Inspect the two candidates again and commit to the one with fewer tokens.
- Days 97-99: Sabotage the other candidate.

If we hang onto two candidates, it is less likely that both of them will accumulate a lot of tokens via bad luck during the second sabotage phase. We benefit from delaying our decision point for as long as possible, while still leaving enough time to deal with our runner-up.